

Contents

1	Zero to Hero Study Handbook: Production-Grade MCP Server Platform	1
1.1	Module 1: Foundations & Architecture	1
1.1.1	1.1 What this project does	1
1.1.2	1.2 Core paradigms and patterns used here	1
1.1.3	1.3 Architecture and component interaction	2
1.2	Module 2: Repository Map	3
1.3	Module 3: Core Execution Flows	3
1.3.1	Flow A: MCP runtime startup (<code>server.py</code>)	3
1.3.2	Flow B: API tool execution (<code>POST /tools</code>)	4
1.3.3	Flow C: Workflow report generation (<code>POST /reports</code>)	4
1.3.4	Flow D: Resource and prompt surfaces	5
1.3.5	Flow E: CLI and Streamlit paths	5
1.3.6	Key persistent data models (SQLite)	5
1.4	Module 4: Setup & Run Guide	6
1.4.1	4.1 Prerequisites (from actual project files)	6
1.4.2	4.2 Environment variables and config	6
1.4.3	4.3 Typical command sequences	6
1.4.4	4.4 Database and seeding/migration behavior	7
1.5	Module 5: Study Plan & Practice Exercises	7
1.5.1	5.1 Ordered self-study sequence	7
1.5.2	5.2 Practice exercises (with solution outlines)	7
1.5.3	5.3 Model answer quality check	9
1.6	Learner Verification Checklist	9

1 Zero to Hero Study Handbook: Production-Grade MCP Server Platform

1.1 Module 1: Foundations & Architecture

1.1.1 1.1 What this project does

This repository implements a production-oriented Model Context Protocol (MCP) platform with multiple surfaces around one shared core:

- MCP server runtimes via `FastMCPAdapter` and `MCPSDKAdapter`.
- REST bridge via FastAPI (`/health`, `/tools`, `/resources`, `/prompts`, `/memory`, `/search`, `/reports`, `/metrics`).
- Operations CLI via Typer (`mcp-server` command).
- Monitoring and administration UI via Streamlit.
- Persistent operational memory in SQLite and semantic memory in ChromaDB.

Main use cases from the actual code:

- Expose discoverable tools/resources/prompts to MCP-compatible clients.
- Execute tool calls with logging, metrics, and auth/RBAC checks.
- Run multi-step report workflows using `WorkflowEngine`.
- Persist conversation/tool/prompt/audit/metrics history for traceability.

1.1.2 1.2 Core paradigms and patterns used here

Definitions first, then where they appear:

- Service composition: A central object wires subsystems together.

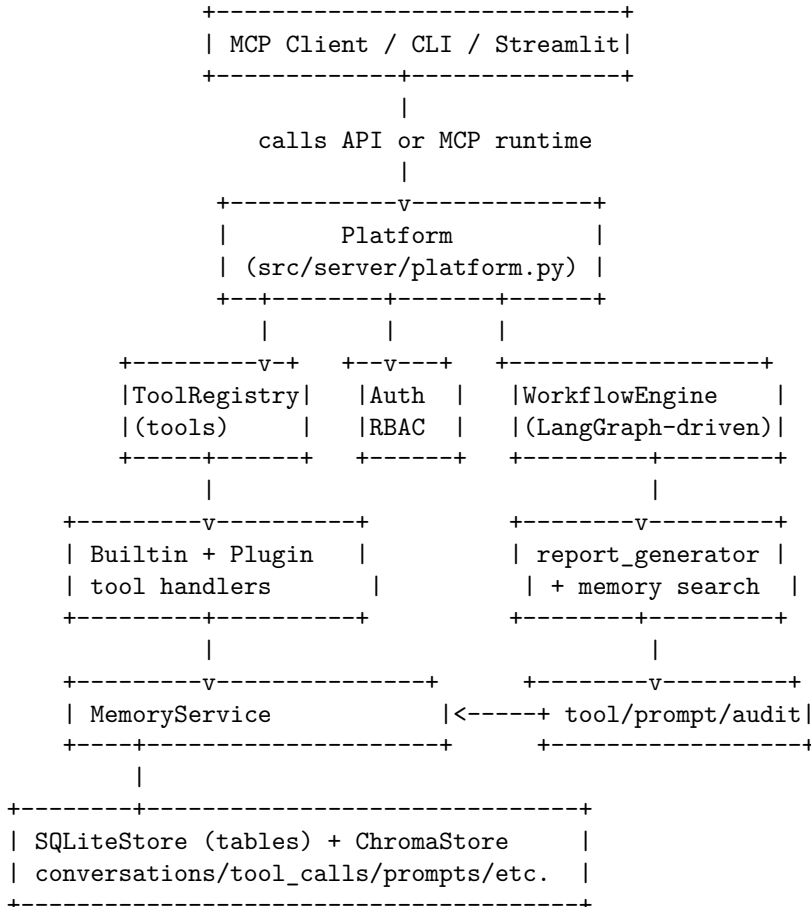
- Here: Platform in `src/server/platform.py` composes auth, memory, registry, adapters, metrics, scheduler, and workflow.
- Adapter pattern: One internal capability model exposed through different protocol runtimes.
 - Here: `FastMCPAdapter` and `MCPSDKAdapter` both map `ToolRegistry` + `ResourceLibrary` + `PromptLibrary` to MCP runtime APIs.
- Registry pattern: Capabilities are registered then looked up by name at runtime.
 - Here: `ToolRegistry.register()`, `ToolRegistry.call()`, `ToolRegistry.list()`.
- Declarative schemas and typed config: Structured data models define contracts.
 - Here: Pydantic models in `src/config/settings.py`, `src/api/schemas.py`, and `src/workflows/state.py`.
- Async execution for I/O-heavy paths: Endpoints and tool handlers are async.
 - Here: async FastAPI handlers in `src/api/app.py`; async tool handlers in `src/tools/builtin.py`.
- Defense-in-depth safety controls:
 - `AuthService` for authentication/authorization.
 - `execute_python_sandboxed()` and `run_shell_whitelisted()` for execution controls.
 - Plugin allowlisting with SHA256 digest checks in `PluginManager`.

1.1.3 1.3 Architecture and component interaction

Core integration points:

- `server.py` is the MCP entrypoint (`main()`), choosing runtime/mode from env and delegating to `Platform.run_mcp_server()`.
- `src/api/main.py` is the API entrypoint (`uvicorn.run(app, host, port)`).
- `app.py` launches Streamlit (`src/ui/streamlit_app/Home.py`).

ASCII main flow (runtime path):



1.2 Module 2: Repository Map

Focus on first-read files for contributors.

File/Directory Path	Primary Responsibility	Key Classes
production_mcp_server/server.py	MCP runtime entrypoint	main()
production_mcp_server/app.py	Streamlit launcher entrypoint	main()
production_mcp_server/src/server/platform.py	Composition root and runtime dispatcher	Platform.f
production_mcp_server/src/config/settings.py	Typed settings model + YAML/env loader	Settings, I
production_mcp_server/configs/default.yaml	Default runtime/config profile	YAML sect
production_mcp_server/src/api/main.py	FastAPI process entrypoint	module-level
production_mcp_server/src/api/app.py	REST routes and auth dependencies	create_api
production_mcp_server/src/api/schemas.py	API request contracts	ToolExecut
production_mcp_server/src/cli/main.py	Typer CLI surface	run, tools,
production_mcp_server/src/auth/security.py	Authentication and authorization	Role, Ident
production_mcp_server/src/tools/base.py	Tool contract model	ToolDefini
production_mcp_server/src/tools/registry.py	Tool registration and dispatch	register()
production_mcp_server/src/tools/builtin.py	19 built-in tool handlers	build_buil
production_mcp_server/src/tools/sandbox.py	Hardened execution helpers	execute_py
production_mcp_server/src/tools/plugins.py	Dynamic plugin loading with integrity checks	PluginMana
production_mcp_server/src/resources/library.py	MCP resource catalog + dynamic URI reader	ResourceDe
production_mcp_server/src/prompts/library.py	Prompt template library	PromptDefi
production_mcp_server/src/workflows/state.py	Workflow state schema	WorkflowSt
production_mcp_server/src/workflows/graph.py	Multi-agent workflow orchestration	WorkflowEr
production_mcp_server/src/memory/service.py	Unified memory API for app subsystems	log_* meth
production_mcp_server/src/memory/sqlite_store.py	SQLAlchemy models and persistence	SQLiteStor
production_mcp_server/src/memory/chroma_store.py	Vector memory adapter	ChromaStor
production_mcp_server/src/monitoring/metrics.py	Runtime metrics collection/logging	MetricsCol
production_mcp_server/src/monitoring/scheduler.py	Periodic indexing/cleanup/report jobs	JobSchedul
production_mcp_server/src/server/fastmcp_adapter.py	FastMCP capability registration	register_c
production_mcp_server/src/server/mcp_sdk_adapter.py	Official MCP SDK registration + fallback	register_c
production_mcp_server/src/ui/streamlit_app/Home.py	Streamlit dashboard landing page	top-level pa
production_mcp_server/src/tests/	Static test references for behavior/contracts	e.g., test_c

1.3 Module 3: Core Execution Flows

1.3.1 Flow A: MCP runtime startup (server.py)

Step-by-step path:

1. `server.main()` resolves config/runtime overrides from env:
 - `config = os.environ.get("MCP_SERVER_CONFIG", "configs/default.yaml")`
 - optional override envs: `MCP_SERVER_RUNTIME`, `MCP_SERVER_TRANSPORT`
2. `Platform.from_config(config)` builds the entire service graph.
3. Runtime/mode override is applied on `platform.settings.transport` if provided.
4. `platform.run_mcp_server()` dispatches:
 - fastmcp runtime: `FastMCPAdapter.register_capabilities()` then `FastMCPAdapter.run(...)`
 - mcp runtime: `MCPSDKAdapter.register_capabilities()` then `MCPSDKAdapter.run(...)`

Important config keys consumed in this flow:

- `transport.runtime`: `fastmcp` | `mcp`
- `transport.mode`: `stdio` | `sse` | `http` | `streamable-http`
- `transport.host`, `transport.port`, `transport.sse_path`, `transport.http_path`

1.3.2 Flow B: API tool execution (POST /tools)

Route and schema:

- Handler: `execute_tool()` in `src/api/app.py`
- Request model: `ToolExecutionRequest`

Exact request shape from `src/api/schemas.py`:

```
{
  "tool_name": "calculator",
  "arguments": {"expression": "(7+5)*3"},
  "session_id": "api"
}
```

Execution chain:

1. `identity_dependency` reads `x-api-key` header.
2. `AuthService.authenticate()` validates key and returns `Identity(api_key, role)`.
3. `require_user()` enforces role threshold (`Role.USER`).
4. If `settings.auth.read_only_mode` is true and tool is mutating (`tool.read_only == false`), `ensure_not_read_only_mode()` blocks request.
5. `platform.call_tool(...)` executes:
 - `ToolRegistry.call(name, payload)` dispatches handler.
 - `MemoryService.log_tool_call(...)` writes to SQLite `tool_calls`.
 - `MetricsCollector.record_tool_latency(...)` writes metric row.
6. Audit event is logged: `memory.log_audit(identity.api_key, "execute_tool", ...)`.
7. Tool result is returned directly.

Observed response shape in real artifact `reports/e2e_api_live_calculator.json`:

```
{"ok": true, "result": 36, "latency_ms": 0}
```

1.3.3 Flow C: Workflow report generation (POST /reports)

Route and schema:

- Handler: `create_report()` in `src/api/app.py`
- Request model: `WorkflowRequest`

Request shape:

```
{"query": "Generate production report after memory lookup"}
```

Execution chain:

1. Endpoint calls `platform.run_workflow(payload.query)`.
2. `WorkflowEngine.run(query)` creates `WorkflowState(query=...)`.
3. Deterministic step sequence executes:
 - `_planner -> _tool_selector -> _mcp_router -> _execution_agent -> _reflection_agent`
4. Branching via `_after_reflection(...)`:
 - fail if no tool output
 - continue if status remains running
 - report otherwise
5. If continue, `_memory_agent` runs semantic retrieval.
6. `_report_agent` calls tool `report_generator` and persists report path in `state.report`.
7. `MemoryService.log_response(...)` stores workflow response metadata.

`WorkflowState` field set (from `src/workflows/state.py`):

- `query`: str
- `plan`: list[str]

- `selected_tools`: `list[str]`
- `tool_outputs`: `list[dict[str, Any]]`
- `memory_context`: `list[dict[str, Any]]`
- `reflection`: `str`
- `report`: `str`
- `steps`: `list[str]`
- `status`: `"running" | "completed" | "degraded" | "failed"`

Observed response keys in `reports/e2e_api_live_workflow.json` include:

- `query`, `plan`, `selected_tools`, `tool_outputs`, `memory_context`, `reflection`, `report`, `steps`, `status`

1.3.4 Flow D: Resource and prompt surfaces

Resource list and read:

- `GET /resources` -> returns `{"resources": [...]}`
- `POST /resources` with `ResourceReadRequest {"uri": "..."}`
- `ResourceLibrary.read(uri)` supports:
 - registered resources (`file://README.md`, `config://default`, `memory://recent-conversations`, `metrics://recent`, `logs://server`)
 - dynamic `csv://<path>`
 - dynamic `sqlite://<query>` placeholder note
 - direct file path read under project root

Prompt list and render:

- `GET /prompts` -> returns prompt metadata list
- `POST /prompts` with:

```
{
  "prompt_name": "documentation_writer",
  "variables": {"topic": "MCP"}
}
```

- `PromptLibrary.render(name, variables)` fills template using built-in role/objective/constraints/expected output plus caller variables.

1.3.5 Flow E: CLI and Streamlit paths

CLI (`src/cli/main.py`):

- `mcp-server run --mode mcp|api|all`: starts runtime path.
- Read-only listing commands (`tools`, `resources`, `prompts`, `monitor`) construct rich tables from platform service methods.
- `report` command executes async workflow via `asyncio.run(platform.run_workflow(query))`.

Streamlit:

- `app.py` points to `src/ui/streamlit_app/Home.py`.
- `ui.service.get_platform()` caches one `Platform` instance via `@lru_cache(maxsize=1)`.
- Pages in `src/ui/streamlit_app/pages/` expose direct read/execute operations (tool run, memory search, logs, metrics charts, settings view).

1.3.6 Key persistent data models (SQLite)

`src/memory/sqlite_store.py` defines table-backed models:

- `Conversation` (`conversations`)
- `ToolCall` (`tool_calls`)

- PromptRun (prompt_runs)
- ResponseRecord (responses)
- MetadataRecord (metadata)
- AuditLog (audit_logs)
- UsageEvent (usage_events)
- CacheEntry (cache_entries)
- MetricRow (metrics)

All tables are created on store initialization via `Base.metadata.create_all(self.engine)`.

1.4 Module 4: Setup & Run Guide

1.4.1 4.1 Prerequisites (from actual project files)

From `pyproject.toml` and docs:

- Python $\geq 3.12, < 3.13$
- uv package manager
- Runtime dependencies include: `fastapi`, `uvicorn`, `fastmcp`, `mcp`, `langgraph`, `streamlit`, `sqlalchemy`, `chromadb`, `httpx`, `typer`, `plotly`, `reportlab`, `duckduckgo-search`, `psutil`
- For PDF export in this handbook workflow: `pandoc` and `xelatex`

1.4.2 4.2 Environment variables and config

`.env` example keys:

- `MCP_SERVER__AUTH__ENABLED`
- `MCP_SERVER__AUTH__READ_ONLY_MODE`
- `MCP_SERVER__TRANSPORT__RUNTIME`
- `MCP_SERVER__TRANSPORT__MODE`
- `MCP_SERVER__MODELS__BASE_URL`
- `NEWS_API_KEY`
- `GITHUB_TOKEN`

Config source and override behavior:

- Base YAML: `configs/default.yaml`
- Env override prefix: `MCP_SERVER__` (parsed by `_apply_env_overrides()` in `src/config/settings.py`)
- Nested override format: `MCP_SERVER__SECTION__FIELD=value`

Important config blocks in `configs/default.yaml`:

- `models`: backend/base URL/default model/supported models
- `transport`: runtime/mode/network paths
- `auth`: enabled/read-only/api key roles
- `memory`: SQLite path, Chroma path, embedding model, top-k
- `plugins`: plugin dir and allowlist manifest
- `logging`, `monitoring`, `scheduler`, `shell`, `external`

1.4.3 4.3 Typical command sequences

Packaging and environment setup (from `README.md` and `pyproject.toml` scripts):

```
cd production_mcp_server
uv venv .venv
source .venv/bin/activate
UV_CACHE_DIR=.uv-cache uv sync --all-groups
```

Main run surfaces:

```

# MCP runtime (via Typer)
UV_CACHE_DIR=.uv-cache uv run mcp-server run --mode mcp --config configs/default.yaml

# API runtime
UV_CACHE_DIR=.uv-cache uv run mcp-server run --mode api --config configs/default.yaml

# Streamlit UI
UV_CACHE_DIR=.uv-cache uv run python app.py

# CLI diagnostics
UV_CACHE_DIR=.uv-cache uv run mcp-server doctor --config configs/default.yaml

```

1.4.4 4.4 Database and seeding/migration behavior

- There are no separate migration scripts in this repo.
- SQLite schema is auto-created by SQLAlchemy models in `SQLiteStore.__init__()`.
- Chroma collection is created/opened by `ChromaStore.__init__()` using `get_or_create_collection(name="mcp_memory")` when enabled.
- Plugin allowlist starts from `configs/plugins_allowlist.yaml` (currently `plugins: {}`), and `PluginManager.generate_allowlist_template()` can populate hashes.

1.5 Module 5: Study Plan & Practice Exercises

1.5.1 5.1 Ordered self-study sequence

Read in this order:

1. `configs/default.yaml` + `src/config/settings.py`
 - Learn the configuration model and env override mechanics.
2. `src/server/platform.py`
 - Understand service composition and runtime dispatch.
3. `src/api/schemas.py` + `src/api/app.py`
 - Learn API contracts and auth-gated execution flow.
4. `src/tools/base.py` + `src/tools/registry.py` + `src/tools/builtin.py`
 - Understand tool metadata, dispatch, and real handlers.
5. `src/memory/service.py` + `src/memory/sqlite_store.py` + `src/memory/chroma_store.py`
 - Understand persistence and semantic retrieval.
6. `src/workflows/state.py` + `src/workflows/graph.py`
 - Understand multi-agent state machine and report generation path.
7. `src/server/fastmcp_adapter.py` + `src/server/mcp_sdk_adapter.py`
 - Understand protocol adapter mapping and fallback behavior.
8. `src/ui/streamlit_app/` and `src/cli/main.py`
 - Understand operational surfaces and UX mapping.

1.5.2 5.2 Practice exercises (with solution outlines)

1.5.2.1 Exercise 1 Question: Trace how `MCP_SERVER__TRANSPORT__MODE=http` affects runtime behavior from load to run.

Solution outline:

- `load_settings()` reads YAML then `_apply_env_overrides()` merges env key into nested payload.
- Pydantic validates into `Settings.transport.mode`.
- `Platform.run_mcp_server()` passes mode into adapter `.run(...)`.
- Adapter chooses transport-specific kwargs (`host/port/sse_path/http_path`).

1.5.2.2 Exercise 2 Question: Explain exactly where and how a successful tool call is persisted.

Solution outline:

- API endpoint `POST /tools` calls `platform.call_tool(...)`.
- `ToolRegistry.call()` executes handler and returns result with `latency_ms`.
- `MemoryService.log_tool_call(...)` writes a `ToolCall` row into `tool_calls` with request/response payload and latency.
- `MetricsCollector.record_tool_latency(...)` writes metric `tool_latency_ms` into `metrics` table.

1.5.2.3 Exercise 3 Question: Why can `POST /metrics` not be called by normal users?

Solution outline:

- Route `GET /metrics` uses dependency `require_admin`.
- `require_admin` calls `platform.auth.authorize(identity, Role.ADMIN)`.
- `authorize()` compares role hierarchy `{read_only:1, user:2, admin:3}` and raises 403 if lower.

1.5.2.4 Exercise 4 Question: List the minimum contract a plugin file must satisfy to load successfully.

Solution outline:

- Plugin file must be in configured `plugins` directory.
- File name must appear in allowlist manifest with matching SHA256 digest.
- Module must export `register_plugin`.
- `register_plugin()` must return a list of `ToolDefinition` objects (or dicts convertible to `ToolDefinition`).

1.5.2.5 Exercise 5 Question: What makes `python_executor` safer than raw Python execution?

Solution outline:

- Static AST validation blocks disallowed imports and names (`_ALLOWED_IMPORTS`, `_BLOCKED_NAMES`).
- Dunder attribute access blocked.
- Isolated subprocess runs `python3 -I` on temp script.
- Resource limits enforced (`RLIMIT_AS`, `RLIMIT_CPU`) and timeout kill path.

1.5.2.6 Exercise 6 Question: In workflow logic, what determines whether `memory_agent` runs?

Solution outline:

- `_after_reflection()` checks `WorkflowState`.
- If no `tool_outputs`: fail.
- If `status == "running"`: continue -> runs `_memory_agent`.
- Else (degraded): route directly to `report_agent`.

1.5.2.7 Exercise 7 Question: Describe what `ResourceLibrary.read()` does for each URI class.

Solution outline:

- Exact URI match in registered resources: return loader output with declared MIME type.
- `csv://...:` loads CSV rows (up to 100) and returns JSON string.
- `sqlite://...:` returns advisory note to use `sqlite_query` tool.
- Fallback file path: reads project-root file if it exists.
- Unknown URI: raises `KeyError`.

1.5.2.8 Exercise 8 Question: Explain how prompt rendering injects defaults and caller variables.

Solution outline:

- `PromptLibrary.get(name)` loads `PromptDefinition`.
- `render()` builds payload with defaults:
 - `role`, `objective`, `constraints` (joined with `;`), `expected_output`
- Caller-provided `variables` are merged and used by `prompt.template.format(**payload)`.

1.5.3 5.3 Model answer quality check

A strong answer should include:

- Exact class/function names from code.
- Correct sequence of call boundaries.
- Accurate request/response or state field names.
- Explicit mention of guards/fallbacks (auth, read-only, plugin hash checks, error return shapes).

1.6 Learner Verification Checklist

Use this checklist to confirm understanding:

- Can you explain `Platform.from_config()` subsystem construction without looking at notes?
- Can you trace `POST /tools` from `x-api-key` to DB write and metric write?
- Can you list all `WorkflowState` fields and when `status` changes?
- Can you explain why `ToolRegistry.call()` may still return `{"ok": false, ...}` for known tools?
- Can you describe how `MCP_SERVER_...` env keys override nested YAML config?
- Can you explain the difference between `FastMCPAdapter` and `MCPSDKAdapter` behavior?
- Can you identify every SQLite table and what event writes to it?
- Can you describe the security controls in `tools/sandbox.py` and `auth/security.py`?
- Can you explain plugin allowlist validation and failure cases?
- Can you map each Streamlit page to its backend service calls?